

1 Fresher Level Node.js Interview Questions

1. What is Node.js and how does it work?

Answer

Node.js is a JavaScript runtime environment that allows JavaScript to run outside the browser, mainly for building backend applications and servers.

It is built on Chrome's V8 JavaScript engine.

How it works

1. User sends request
2. Node.js receives request
3. Event Loop processes request
4. Non-blocking I/O handles operations
5. Response returned to client

Reason

Node.js uses event-driven, non-blocking architecture, so it can handle many requests efficiently.

2. What is NPM?

Answer

NPM stands for Node Package Manager.

It is used to install, manage, and share libraries (packages) in Node.js projects.

Example:

```
npm install express
```

Reason

Instead of writing everything from scratch, developers use ready-made packages.

3. What are modules in Node.js?

Answer

Modules are **reusable pieces of code** that can be imported into other files.

Example:

```
const fs = require('fs');
```

Types of modules

1. **Core modules** (fs, http) → (built-in)
2. **Local modules** → Local modules are **modules created by you inside your project**.
3. **Third-party modules** → Third-party modules are **packages created by other developers and installed using NPM**. They provide **extra functionality**.

Examples

- express → Web framework
- mongoose → MongoDB ODM
- cors → Cross-origin requests
- nodemon → Auto restart server

Reason

Modules make code **organized and reusable**.

4. What is the purpose of require() in Node.js?

Answer

require() is used to **import modules**.

Example:

```
const http = require('http');
```

Reason

It allows Node.js files to **share code between modules**.

5. What is module.exports?

Answer

module.exports is used to **export functions or variables from a module**.

Example

```
function add(a,b){  
  return a+b;  
}  
  
module.exports = add;
```

Importing

```
const add = require('./math');
```

Reason

It allows **communication between files**.

6. What is REPL in Node.js?

Answer

REPL stands for

Read

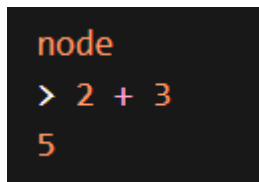
Evaluate

Print

Loop

It is an **interactive Node.js shell**.

Example

A screenshot of the Node.js REPL (Read-Evaluate-Print-Loop) interface. The prompt 'node' is shown in orange. Below it, the input '> 2 + 3' is shown in orange, and the output '5' is shown in orange. The background is dark.

Reason

It helps developers **test code quickly**.

7. What is V8 Engine in Node.js?

Answer

V8 is **Google's JavaScript engine** used in Chrome and Node.js.

It **converts JavaScript into machine code**.

Reason

This makes JavaScript execution **very fast**.

8. Why is Node.js single-threaded?

Single Main Thread in Node.js:

Node.js runs all JavaScript code on **one main thread**, which processes tasks one by one using the **event loop**.

Simple explanation

- There is **one thread executing JavaScript**
- It handles requests **sequentially**

- Time-consuming tasks (file read, database, API calls) are handled **asynchronously in the background**
- A thread is a path that a program uses to run instructions.

9. If Node.js is single-threaded, how does it handle concurrency?

Answer

Node.js handles concurrency using:

- 1 Event Loop
- 2 Callback Queue
- 3 Non-blocking I/O

Example

```
setTimeout(()=>console.log("Hello"),2000);
console.log("World");
```

Output

World
Hello

Reason

Tasks are executed **asynchronously**.

Simple Explanation

When a request comes:

- 1 Request arrives at **Node.js server**
- 2 **Event Loop** receives the request
- 3 If the task is **slow (file, DB, API)** → it is sent to **background worker threads (libuv)**
- 4 Main thread continues handling **other requests**
- 5 When the task finishes → callback is added to the **callback queue**
- 6 Event loop executes the callback

10. What is the difference between synchronous and asynchronous functions?

Synchronous

Runs line by line. ,Example

```
console.log("A");
console.log("B");
```

Output

A

B

Asynchronous

Runs without blocking execution.

Example

```
setTimeout(()=>console.log("A"),1000);  
console.log("B");
```

Output

B

A

11. What is control flow in Node.js?

Control flow in Node.js is the way a program controls the order in which functions and tasks are executed.

In Node.js, many operations are **asynchronous**, so control flow helps decide **which task runs first, next, and after completion**.

Why Control Flow is Important

Because Node.js handles many **asynchronous operations** like:

- File reading
- Database queries
- API requests

Control flow ensures that **tasks execute in the correct order**.

Tools Used for Control Flow

1 Callbacks

A function that runs **after another function finishes**.

2 Promises

Promises help manage asynchronous operations more cleanly.

3 Async / Await

Makes asynchronous code look **like synchronous code**.

4 Async Library

The **async library** provides utilities to manage complex asynchronous flows like:

- `async.parallel()` -> Runs **multiple asynchronous functions at the same time**.
- `async.series()` -> Runs asynchronous functions **one after another (sequentially)**.
- `async.waterfall()` -> Runs functions **in sequence**, where **each function passes its result to the next function**.

Execution Flow

Task1 → result → Task2 → result → Task3 → final result

These functions come from the **Async.js** library and are used to **control the flow of asynchronous functions in Node.js**.

They were very popular before **Promises** and **async/await**, but interviewers still ask them.

Reason

Helps manage multiple asynchronous tasks.

12. What are the different types of API functions in Node.js?

Two types

1 Blocking (Synchronous)

2 Non-Blocking (Asynchronous)

Example

Blocking

fs.readFileSync() -> `fs.readFileSync()` reads a file completely and blocks the program execution until the file reading is finished.

That means **Node.js will wait** until the file is read.

Non-Blocking

`fs.readFile()` -> `fs.readFile()` reads a file asynchronously without blocking the Node.js event loop.

This means **Node.js continues executing other code while the file is being read**.

13. What are commonly used Node.js timing functions?

- 1 **setTimeout()** -> `setTimeout()` executes a function **once after a specified delay**.
- 2 **setInterval()** -> `setInterval()` **executes** a function **repeatedly** at a fixed interval.
- 3 **setImmediate()** -> `setImmediate()` executes a function **immediately after the current event loop cycle completes**.
- 4 **process.nextTick()** -> `process.nextTick()` runs a function immediately after the current code finishes, before Node.js handles timers or other async tasks.

14. What are the main disadvantages of Node.js?

- 1 Not suitable for CPU heavy tasks

Node.js works on a **single main thread**.

If a task requires **heavy computation**, it blocks the event loop.

Example

Tasks like:

- Image processing

Blocking the event loop means Node.js cannot process any other tasks or requests until the current task finishes.

Because Node.js has **one main thread**, if that thread is busy with a heavy task, everything else must **wait**.

- 2 Callback complexity

When many asynchronous operations depend on each other, callbacks become **nested and messy**.

Example

```
fs.readFile("file1.txt", (err, data1) => {
  fs.readFile("file2.txt", (err, data2) => {
    fs.readFile("file3.txt", (err, data3) => {
      console.log(data1, data2, data3);
    });
  });
});
```

This structure becomes hard to:

- read
- maintain

- debug

This problem is called **Callback Hell**.

Modern Solution

Use:

- **Promises**
- **async/await**

15. How do you manage packages in Node.js?

Using **NPM** or **Yarn**.

Main commands:

Install

npm install express

Update

npm update

Remove

npm uninstall express

Dependencies stored in:

package.json

16. What is package.json?

package.json is the **configuration file of a Node project**.

It contains:

- dependencies
- scripts
- project name
- version


```
{
  "name": "myapp",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

17. Can you access DOM in Node.js?

✗ No.

Node.js runs **outside the browser**, so there is **no DOM** or **window object**.

18. How to create a simple HTTP server?

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(3000);
```

Intermediate Node.js Interview Questions

19. What is callback hell?

Callback hell is when many callbacks are nested inside each other, making the code difficult to read and maintain.

```
function orderFood(callback) {  
  console.log("1. Order received");  
  callback();  
}  
  
function prepareFood(callback) {  
  console.log("2. Food is being prepared");  
  callback();  
}  
  
function deliverFood(callback) {  
  console.log("3. Food delivered");  
  callback();  
}  
  
orderFood(() => {  
  prepareFood(() => {  
    deliverFood(() => {  
      console.log("4. Enjoy your meal!");  
    });  
  });  
});
```

Where Function Is Called

This is the **main calling line**:

```
orderFood(() => {  
  prepareFood(() => {  
    deliverFood(() => {  
      console.log("4. Enjoy your meal!");  
    });  
  });  
});
```

Here we call **orderFood()** and pass a **callback function**.

Step-by-Step Execution

Step 1 orderFood() runs

```
console.log("1. Order received");  
callback();
```

Output

1. Order received

Then **callback()** executes.

Step 2

Callback contains:

```
prepareFood(() => {  
  deliverFood(() => {  
    console.log("4. Enjoy your meal!");  
  });  
});
```

So now **prepareFood()** runs.

Output

2. Food is being prepared

Then **callback()** runs again.

Step 3

Callback contains:

```
deliverFood(() => {  
  console.log("4. Enjoy your meal!");  
});
```

So **deliverFood()** runs

Output

3. Food delivered

Then **callback()** runs.

Step 4

Final callback runs

4. Enjoy your meal!

Final Output

1. Order received
2. Food is being prepared
3. Food delivered
4. Enjoy your meal!

Important Understanding

Inside each function:

`callback();`

means

👉 **Run the next function that was passed.**

A **callback function** is a function passed as an argument to another function and executed later using `callback()`.

20. What are promises?

Promises represent **future completion of async operations**.

States:

- ❶ Pending
- ❷ Fulfilled
- ❸ Rejected

Example

```
fetchData()  
.then(data=>console.log(data))  
.catch(err=>console.log(err));
```

21. How can we use async/await?

Example

```
async function test(){  
  let data = await fetchData();  
  console.log(data);  
}
```

Reason

It makes asynchronous code look **synchronous and readable**.

22. What is middleware?

Middleware are **functions executed between request and response**.

Example (Express)

```
app.use((req,res,next)=>{  
  console.log("Request received");  
  next();  
});
```

app.use() is a method in **Express.js** used to **register middleware functions**.

app.use() is used to add middleware in an Express application.

Middleware executes **before the request reaches the route handler**.

23. What are Node.js buffers?

A Buffer is **a temporary memory area used to store binary data while it is being transferred or processed**.

Node.js uses **buffers** because **binary data cannot be handled directly by JavaScript strings**.

Simple Example

```
const buffer = Buffer.from("Hello");  
  
console.log(buffer);
```

Output

<Buffer 48 65 6c 6c 6f>

Explanation:

Hello → converted to binary → stored in buffer

Each number represents ASCII / binary value.

Why Buffers Are Needed

When Node.js reads a file or receives data from the network, the data comes as binary chunks.

Example:

File → binary data → stored in Buffer → processed

Without buffers, Node.js cannot efficiently handle binary data.

Used in

- file system
- streams
- TCP data

24. What are streams?

Streams allow data to be read or written continuously in small chunks instead of loading the whole data into memory.

Simple Example

Imagine a large 1GB file.

Without Stream

File → load entire 1GB into memory → process

Problem:

- uses large memory
- slow

With Stream

File → read small chunk → process → next chunk → process

Memory usage is very small.

```
const fs = require("fs");

const stream = fs.createReadStream("file.txt");

stream.on("data", (chunk) => {
  console.log("Received chunk:", chunk.toString());
});
```

What happens?

file.txt

↓

chunk1

chunk2

chunk3

Node.js reads small pieces of the file.

Real-Life Example

Think of drinking water from a pipe.

Without Stream

Fill entire tank → drink water

With Stream

Drink water directly from pipe continuously

Streams work like the pipe.

Types of Streams in Node.js

Stream Type Meaning

Readable Read data

Writable Write data

Duplex Read and write

Transform Modify data while streaming

25. What is piping in Node.js?

Piping is a method used to transfer data from a readable stream to a writable stream using `.pipe()`.

Syntax

```
readableStream.pipe(writableStream);
```

Example

```
const fs = require("fs");
```

```
const readStream = fs.createReadStream("input.txt");  
const writeStream = fs.createWriteStream("output.txt");
```

```
readStream.pipe(writeStream);
```

How It Works

input.txt → readStream → pipe() → writeStream → output.txt

Data flow:

chunk1 → write

chunk2 → write

chunk3 → write

The file is **copied chunk by chunk**.

Why Piping is Useful

Without piping:

Read entire file → store in memory → write file

With piping:

Read small chunk → send to destination → next chunk

Advantages:

- Faster
- Uses less memory
- Handles large files easily

26. Difference between process.nextTick() and setImmediate()

process.nextTick()

Runs **before event loop continues**.

setImmediate()

Runs **in the next event loop iteration**.

27. What is fork in Node.js?

fork() creates a new Node.js process that runs in parallel with the main process.

It helps handle **CPU-heavy tasks without blocking the main thread**.

Why fork() is needed

Node.js runs on **one main thread**.

If a heavy task runs:

Main Thread → Heavy Task → Server stops responding

Using fork():

Main Thread → sends task → Child Process

Main Thread → continues handling requests

Syntax

```
const { fork } = require("child_process");
```

```
fork("file.js");
```

Simple Example

main.js

```
const { fork } = require("child_process");
```

```
const child = fork("worker.js");
```

```
console.log("Main process running");
```

worker.js

```
console.log("Child process running");
```

Output

Main process running

Child process running

Two processes are running **independently**.

Communication Between Processes

Parent and child can **send messages**.

main.js

```
const { fork } = require("child_process");
```

```
const child = fork("worker.js");
```

```
child.send("Hello child");
```

worker.js

```
process.on("message", (msg) => {  
  console.log("Message from parent:", msg);  
});
```

Output

Message from parent: Hello child

28. What is CORS?

CORS stands for:

Cross-Origin Resource Sharing

It allows APIs to accept requests from **different domains**.

```
const cors = require("cors");  
app.use(cors());
```

1 First Understand This Rule

Browsers follow a security rule called **Same-Origin Policy**.

This rule says:

A website can only request data from the **same origin** (same domain).

2 What is Origin?

Origin =

Protocol + Domain + Port

Example:

Frontend → http://localhost:3000

Backend → http://localhost:5000

Ports are different → **Different origin**

3 What Problem Happens

Suppose your **React frontend** calls your **Node.js API**.

```
fetch("http://localhost:5000/users")
```

But your frontend is running on:

http://localhost:3000

Browser sees:

3000 → trying to access → 5000

Browser says ❌

Blocked by CORS policy

4 Why Browser Blocks It

For **security**.

Example attack:

You login to bank.com

↓

Malicious website tries to steal bank data

↓

Browser blocks request

This protection is **CORS**.

5 How We Fix It

We tell the server:

Allow requests from this website

Example in **Express.js**

```
const express = require("express");
```

```
const cors = require("cors");
```

```
const app = express();
```

```
app.use(cors());
```

Now browser sees:

Server allows request

So request works.

6 Visual Explanation

Without CORS:

Frontend (3000)

↓ request

Backend (5000)

Browser ❌ BLOCKS

With CORS:

Frontend (3000)

↓ request

Backend (5000)

↓

Server says: Allowed

Browser ✅ allows

7 Real MERN Example

React App → localhost:3000

Node API → localhost:5000

React calls API:

```
fetch("http://localhost:5000/products")
```

If CORS not enabled:

CORS Error

If enabled:

Data received successfully

8 Very Short Interview Answer

CORS allows a server to accept requests from a different origin (domain, protocol, or port).

29. What is WebSocket?

WebSocket allows **real-time communication** between client and server.

Example uses

- Chat apps
- Live notifications
- Online gaming

Libraries

socket.io

ws

30. What is the difference between spawn() and fork()?

spawn()

- launches new process
- runs any command

fork()

- creates new **Node.js process**
 - communication via IPC
-

3 Advanced Node.js Interview Questions

31. What is Event Loop?

The **event loop** manages asynchronous operations.

Steps

- 1 Call Stack
 - 2 Web APIs
 - 3 Callback Queue
 - 4 Event Loop executes tasks
-

32. What is EventEmitter?

Node.js uses **event-driven architecture**.

Example

```
const EventEmitter = require('events');
const event = new EventEmitter();
```

```
event.on('hello',()=>{
  console.log("Hello event triggered");
});
```

```
event.emit('hello');
```

33. What is clustering in Node.js?

Clustering allows Node.js to **use multiple CPU cores**.

Example

```
const cluster = require('cluster');
```

Reason

Improves **performance and scalability**.

34. What is thread pool in Node.js?

Node.js uses a **thread pool managed by libuv**.

Used for:

- file system
- DNS
- crypto

Default size

4 threads

35. What are worker threads?

Worker threads allow **parallel execution of JavaScript code**.

Example

```
const { Worker } = require('worker_threads');
```

Used for **CPU intensive tasks**.

36. How to measure performance of async operations?

Using

```
console.time()  
console.timeEnd()
```

Example

```
console.time("test");
```

```
await fetchData();
```

```
console.timeEnd("test");
```

37. What is Reactor Pattern in Node.js?

Reactor Pattern is a **design pattern used in Node.js event loop**.

Steps

- 1 Request received
- 2 Event demultiplexer waits for events
- 3 Event handler executes callback

38. What is WASI?

WASI = **WebAssembly System Interface**

It allows **WebAssembly** programs to run outside browsers.

Used to improve **performance and portability**.

39. How to connect Node.js with MongoDB?

Using **mongoose**.

Example

```
const mongoose = require('mongoose');
```

```
mongoose.connect("mongodb://localhost:27017/mydb")  
.then(()=>console.log("Connected"));
```